Représentation des Entiers Signés et Arithmétique Binaire : Le Complément à Deux

Thème principal : Codage des entiers relatifs, algorithme du Complément à Deux et arithmétique modulaire dans l'UAL

Synthèse exécutive : Ce cours aborde la problématique fondamentale du stockage et du calcul des nombres entiers négatifs au niveau matériel. Pour éviter d'intégrer des circuits électroniques de soustraction distincts (très coûteux en transistors, en surface de silicium et en consommation énergétique), l'Unité Arithmétique et Logique (UAL) réutilise le circuit d'addition binaire en transformant toute soustraction A - B en une addition A + (-B). La convention internationale du Complément à Deux (C2) permet ce codage : elle offre un algorithme simple (C2 = C1 + 1), garantit une représentation unique de la valeur zéro (éliminant l'anomalie du +0 et -0), fonctionne selon les règles de l'arithmétique modulaire (modulo 2^n) et permet une gestion rigoureuse de la retenue sortante (Carry) et du dépassement de capacité (Overflow).

1. Problématique matérielle : La soustraction sans circuit soustracteur

Au niveau de l'architecture physique d'un microprocesseur, concevoir un circuit électronique distinct spécifiquement dédié à la soustraction binaire (un soustracteur matériel) doublerait inutilement la complexité de l'UAL par rapport à un simple additionneur. Pour contourner cet obstacle et optimiser la surface de silicium, les architectes matériels ramènent toute opération de soustraction A - B sous la forme d'une addition A + (-B). La difficulté centrale consiste à définir une représentation binaire pour les entiers négatifs (-B) qui permette à un additionneur binaire conventionnel d'effectuer le calcul bit à bit de manière transparente et d'obtenir le résultat mathématique exact. Les conventions historiques présentaient de sévères limitations : - Bit de Signe + Magnitude : Le MSB donne le signe (0 = +, 1 = -) et les autres bits donnent la valeur absolue. Problème : le calcul d'addition donne des résultats faux sans logique de correction complexe, et il existe deux zéros (+0 = 00000000 et -0 = 10000000). - Complément à Un (C1) : Obtenu en inversant tous les bits. Problème : nécessite d'ajouter la retenue sortante au résultat (rebouclage) et conserve le problème du double zéro.

- Unité Arithmétique et Logique (UAL / ALU) : Composant électronique interne au processeur chargé d'exécuter les opérations logiques et mathématiques fondamentales.
- Optimisation par additionneur binaire : Réutilisation du circuit d'addition matériel pour réaliser les soustractions afin de réduire le nombre de transistors, la consommation électrique et le coût de fabrication de la puce.
- Transformation de la soustraction : Équivalence arithmétique fondamentale transformant toute soustraction A - B en une addition binaire A + (-B).
- Bit de poids fort (MSB - Most Significant Bit) : Le bit situé le plus à gauche d'un mot binaire, servant d'indicateur de signe dans les conventions signées (0 pour positif ou nul, 1 pour strictement négatif).

Cas concret enseignant : *Analogie du compteur kilométrique mécanique à rouleaux d'une voiture ancienne (limité à 4 chiffres, variant de 0000 à 9999) : si le véhicule affiche 0000 km et effectue une marche arrière de 1 km, le rouleau mécanique recule et affiche 9999. Dans la mécanique de cet engrenage cyclique, 9999 est le représentant physique direct de -1. En binaire sur 8 bits, le principe est identique : si l'on part de 00000000 et qu'on soustrait 1, le registre bascule à 11111111, qui constitue la représentation matérielle exacte de l'entier relatif -1.*

2. Algorithme du Complément à Deux et propriétés structurelles

Pour déterminer la représentation binaire d'un nombre entier négatif (-X) sur un mot de n bits en Complément à Deux, l'algorithme s'exécute impérativement en trois étapes séquentielles : 1. Convertir la valeur absolue X en binaire naturel sur n bits (en complétant à gauche par des zéros si nécessaire). 2. Inverser l'ensemble des bits pour obtenir le Complément à 1 (C1). 3. Ajouter 1 au résultat obtenu pour calculer le Complément à 2 (C2 = C1 + 1). Propriétés clés du Complément à Deux : - Unicité du zéro : En calculant C2(0) sur 8 bits : 00000000 -> C1 =

11111111 -> C2 = 11111111 + 1 = (1)00000000. Le 9ème bit retenu sort du registre et se trouve rejeté par le matériel, ce qui renvoie le mot 00000000. Le zéro possède donc une représentation strictement unique. - Asymétrie de la plage : Sur 8 bits signés, l'intervalle s'étend de -128 à +127. Le nombre -128 (10000000) ne possède pas de pendant positif (+128) codable sur 8 bits.

- Complément à 1 (C1) : Inversion logique bit à bit de l'intégralité du mot binaire (transformation des 0 en 1 et des 1 en 0).
- Complément à 2 (C2) : Algorithme définissant l'opposé d'un nombre binaire via la formule C2(X) = C1(X) + 1.
- Unicité du Zéro : Propriété structurelle majeure éliminant l'anomalie du double zéro (+0 et -0) des conventions historiques.
- Plage de représentation : Sur n bits en C2, l'intervalle des valeurs représentables s'étend strictement de -2^(n-1) à +(2^(n-1) - 1).

Cas concret enseignant : *Démonstration au tableau de l'unicité du zéro sur 8 bits : C2(0) = C1(00000000) + 1 = 11111111 + 1 = (1)00000000. Le bit de débordement 1 sort du registre 8 bits et est ignoré par le matériel, ramenant la valeur à 00000000 sans créer d'artefact '-0'.*

3. Différence fondamentale entre Retenue (Carry) et Dépassement de capacité (Overflow)

Il est primordial de ne pas confondre le drapeau de retenue sortante (Carry flag C) et le drapeau de dépassement de capacité (Overflow flag V). - Carry (Retenue sortante) : Lors de l'exécution d'une soustraction A - B sous la forme A + (-B), la génération d'un 9ème bit de retenue (sur un registre 8 bits) est le comportement mathématique normal de l'arithmétique modulaire (modulo 2^n). Ce bit sort du registre et n'altère pas le résultat conservé dans le registre. - Overflow (Dépassement de capacité) : Il s'agit d'une erreur d'interprétation arithmétique lorsque la valeur théorique calculée dépasse l'intervalle représentable [-2^(n-1) ; +(2^(n-1) - 1)]. Conditions d'Overflow : 1. L'addition de deux nombres de signes opposés (un positif et un négatif) ne peut JAMAIS provoquer d'overflow. 2. L'overflow survient exclusivement si l'on additionne deux nombres positifs et que l'on obtient un résultat négatif (MSB = 1), ou deux nombres négatifs et que l'on obtient un résultat positif (MSB = 0).

- Drapeau Carry (C / Retenue sortante) : Retenue générée au-delà du bit de poids fort (MSB). Lors d'une soustraction A + (-B) en C2, cette retenue est normale et ignorée.
- Drapeau Overflow (V / Débordement de capacité) : Drapeau d'erreur de l'UAL indiquant que le résultat d'un calcul dépasse la plage représentable du registre.
- Arithmétique modulaire : Fonctionnement des registres de taille fixe opérant implicitement modulo 2^n.
- Règle de détection d'Overflow : Un overflow survient si et seulement si l'on additionne deux nombres de même signe et que l'on obtient un résultat de signe opposé.

Cas concret enseignant : *Exemple piège d'évaluation donné par le professeur : Additionner +70 et +60 sur 8 bits signés. Les deux opérandes sont positifs, leur somme théorique est +130. Or, la limite maximale sur 8 bits signés est +127. L'addition binaire (01000110 + 00111100) produit 10000010_2, dont le MSB vaut 1 (ce qui équivaut à -126 en décimal). L'UAL détecte que deux positifs ont donné un résultat négatif et lève immédiatement le drapeau Overflow (V=1).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le bit de retenue tout à gauche (la retenue sortante / 9ème bit) disparaît-il simplement lors du calcul d'une addition en complément à deux ?
Réponse du professeur : Les registres du processeur ont une taille physique fixe (par exemple 8 bits, 16 bits ou 64 bits). Tout bit généré au-delà de cette taille ne peut pas être stocké dans le registre de résultat. L'additionneur binaire réalise une opération d'arithmétique modulaire modulo 2^n. Ce 9ème bit sort du registre de calcul et va alimenter le drapeau de retenue (Carry flag C) dans le registre d'état (FLAGS) du processeur, mais il est ignoré pour la valeur finale conservée dans le registre.

Question étudiant : Comment le processeur sait-il si la suite de bits '11111111' représente le nombre +255 (en entier non signé) ou le nombre -1 (en entier signé C2) ?
Réponse du professeur : Le processeur ne le sait pas et ne cherche pas à le savoir. Dans la mémoire et les registres, il n'y a que des 0 et des 1 sans aucune étiquette de type. C'est l'instruction du programme (l'instruction assembleur choisie par le compilateur ou le type de variable déclaré par le programmeur comme uint8_t vs int8_t) qui détermine comment interpréter ce motif binaire. Si le programmeur utilise une instruction d'affichage non signée, 11111111 affiche 255 ; s'il utilise une instruction signée, 11111111 affiche -1.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuez la soustraction arithmétique 35 - 18 en la convertissant en une addition en Complément à Deux : 35 + (-18). Détailler chaque étape de conversion, poser l'addition binaire et valider le résultat décimal final.

Démarche de résolution :
Étape 1 : Conversion de l'entier positif +35 en binaire naturel sur 8 bits. 35 = 32 + 2 + 1 = 00100011_2. Étape 2 : Codage de l'entier négatif -18 en Complément à Deux (C2). a) Représentation de +18 en binaire sur 8 bits : 18 = 16 + 2 = 00010010_2. b) Calcul du Complément à 1 (C1, inversion de tous les bits) : 11101101_2. c) Calcul du Complément à 2 (C2 = C1 + 1) : 11101101 + 1 = 11101110_2. Le mot binaire 11101110 représente l'entier -18 sur 8 bits. Étape 3 : Addition binaire des deux opérandes (+35) + (-18). 0 0 1 0 0 0 1 1 (+35) + 1 1 1 0 1 1 1 0 (-18) ---------- (1) 0 0 0 1 0 0 0 1 Étape 4 : Gestion de la retenue sortante. Le 9ème bit tout à gauche (1) dépasse la capacité du registre 8 bits. Il est placé dans le drapeau Carry (C=1) et rejeté du résultat final stocké dans le registre.

```
/**
 * @file exercice_c2.cpp
 * @brief Demonstration de la soustraction par addition en C2 (R1.02)
 * @author Matei de Saint-Leger
 * @date 13/11/2026
 */

#include <iostream>
#include <cstdint>
#include <bitset>

using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int8_t operandeA = 35;
    int8_t operandeB = -18;
    int8_t resultat = 0;

    // TRAITEMENTS
    resultat = static_cast<int8_t>(operandeA + operandeB);

    cout << "35  en binaire (8 bits) : " << bitset<8>(operandeA) << endl;
    cout << "-18 en binaire (8 bits) : " << bitset<8>(operandeB) << endl;
    cout << "---------------------------------" << endl;
    cout << "Somme (35 + (-18))      : " << bitset<8>(resultat) << endl;
    cout << "Valeur decimale        : " << static_cast<int>(resultat) << endl;

    return 0;
}
```

Résultat : Le résultat binaire conservé dans le registre 8 bits est 00010001_2. Sa conversion en décimal donne : 2^4 + 2^0 = 16 + 1 = +17. La soustraction arithmétique théorique (35 - 18 = 17) est parfaitement validée par l'UAL.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des valeurs décimales représentables par un entier signé codé sur 8 bits en Complément à Deux ?

R : La plage s'étend de -128 à +127, soit de -2^7 à +(2^7 - 1).

Q : Quel est le codage binaire sur 8 bits de l'entier relatif -5 en Complément à Deux ?

R : 11111011_2. (Détail : +5 = 00000101, C1 = 11111010, C2 = C1 + 1 = 11111011).

Q : Quelle condition exacte provoque l'activation du drapeau de dépassement de capacité (Overflow / V=1) lors d'une addition binaire signée ?

R : L'overflow se produit si et seulement si l'on additionne deux nombres de MÊME signe et que le résultat binaire obtenu présente un signe OPPOSÉ.